Advanced Optimization

Chapter 7 — Large-Scale Unconstrained Optimization

Hongseok Kim

Department of Electronic Engineering, Sogang University

Fall 2026

Reading: Nocedal & Wright, *Numerical Optimization*, 2nd ed., Chapter 7

Large-scale unconstrained optimization

Thousands or millions of variables: the storage and computational cost of the algorithm must be kept at a tolerable level.

Section	Approach	Idea
§7.1	Inexact Newton	approximate Newton step by CG or Lanczos
§7.2	Limited-memory quasi-Newton	a few vectors of length n instead of a matrix
§7.3	Sparse quasi-Newton	B_k with the sparsity pattern of the Hessian
§7.4	Partial separability	f is a sum of element functions

- If a sparse factorization of the Hessian is affordable, Newton methods based on sparse factorizations are effective.
- Nonlinear CG (§5.2) applies to large problems without modification.

Inexact Newton methods

The Newton step p_k^N solves the symmetric $n \times n$ linear system

$$\nabla^2 f_k \, p_k^N = -\nabla f_k. \tag{7.1}$$

Solve (7.1) approximately by CG or Lanczos. Residual of the inexact Newton step p_k:

$$r_k = \nabla^2 f_k \, p_k + \nabla f_k. \tag{7.2}$$

Terminate the iterative solver when

$$\|r_k\| \leq \eta_k \|\nabla f_k\|, \tag{7.3}$$

where $\{\eta_k\}$, with $0 < \eta_k < 1$ for all k, is the **forcing sequence**. No factorization of $\nabla^2 f_k$ is needed.

Theorem 7.1: local convergence

Theorem 7.1

Suppose that $\nabla^2 f(x)$ exists and is continuous in a neighborhood of a minimizer x^*, with $\nabla^2 f(x^*)$ positive definite. Consider the iteration $x_{k+1} = x_k + p_k$ where p_k satisfies (7.3), and assume that $\eta_k \leq \eta$ for some constant $\eta \in [0, 1)$. Then, if the starting point x_0 is sufficiently near x^*, the sequence $\{x_k\}$ converges to x^* and satisfies

$$\|\nabla^2 f(x^*)(x_{k+1} - x^*)\| \leq \hat{\eta} \, \|\nabla^2 f(x^*)(x_k - x^*)\|, \tag{7.4}$$

for some constant $\hat{\eta}$ with $\eta < \hat{\eta} < 1$.

Local convergence only requires that η_k is bounded away from 1.
The book gives an informal derivation, not a rigorous proof. **Read:** pp. 166–167.

Theorem 7.2: the forcing sequence determines the rate

From the derivation of Theorem 7.1:
$$\frac{\|\nabla f_{k+1}\|}{\|\nabla f_k\|} \leq \eta_k + o(1). \tag{7.8}$$

Theorem 7.2

Suppose that the conditions of Theorem 7.1 hold, and assume that the iterates $\{x_k\}$ generated by the inexact Newton method converge to x^*. Then the rate of convergence is superlinear if $\eta_k \to 0$. If in addition, $\nabla^2 f(x)$ is Lipschitz continuous for x near x^* and if $\eta_k = O(\|\nabla f_k\|)$, then the convergence is quadratic.

- $\eta_k = \min\left(0.5, \sqrt{\|\nabla f_k\|}\right)$: superlinear. $\eta_k = \min(0.5, \|\nabla f_k\|)$: quadratic.
- The results are local and assume the unit step $\alpha_k = 1$. **Read:** pp. 167–168.

Line search Newton–CG (truncated Newton)

Apply the CG method to the Newton equations, written as

$$B_k p = -\nabla f_k, \qquad B_k = \nabla^2 f_k, \tag{7.9}$$

and stop by a test of the form (7.3). Inner CG iteration: directions d_j, iterates z_j.

CG requires a positive definite matrix, and $\nabla^2 f_k$ may have negative eigenvalues. Differences from Algorithm 5.2:

- the starting point is $z_0 = 0$;
- a positive tolerance ϵ_k lets CG terminate at an inexact solution;
- the negative curvature test $d_j^T B_k d_j \leq 0$ ensures that p_k is a descent direction. If it holds at $j = 0$, the method returns $p_k = -\nabla f_k$.

Algorithm 7.1 (line search Newton–CG)

Given initial point x_0

for $k = 0, 1, 2, \ldots$

define tolerance $\epsilon_k = \min\left(0.5, \sqrt{\|\nabla f_k\|}\right) \|\nabla f_k\|$

set $z_0 = 0$, $r_0 = \nabla f_k$, $d_0 = -r_0 = -\nabla f_k$

for $j = 0, 1, 2, \ldots$

if $d_j^T B_k d_j \leq 0$: **if** $j = 0$ **return** $p_k = -\nabla f_k$; **else return** $p_k = z_j$

set $\alpha_j = r_j^T r_j / d_j^T B_k d_j$; $z_{j+1} = z_j + \alpha_j d_j$; $r_{j+1} = r_j + \alpha_j B_k d_j$

if $\|r_{j+1}\| < \epsilon_k$ **return** $p_k = z_{j+1}$

set $\beta_{j+1} = r_{j+1}^T r_{j+1} / r_j^T r_j$; $d_{j+1} = -r_{j+1} + \beta_{j+1} d_j$

set $x_{k+1} = x_k + \alpha_k p_k$, where α_k satisfies the Wolfe, Goldstein, or Armijo backtracking conditions (using $\alpha_k = 1$ if possible)

Algorithm 7.1: weakness, and Hessian-free implementation

- **Weakness.** When $\nabla^2 f_k$ is nearly singular, the direction can be long and of poor quality: many function evaluations in the line search, small reduction in f. The trust-region Newton–CG method handles this more effectively.
- **Hessian-free.** The algorithm needs only products $\nabla^2 f_k d$, not the matrix. With a small differencing interval h:

$$\nabla^2 f_k d \approx \frac{\nabla f(x_k + hd) - \nabla f(x_k)}{h}. \tag{7.10}$$

The accuracy is $O(h)$. Price: one new gradient evaluation per CG iteration. Automatic differentiation (Chapter 8) gives $\nabla^2 f_k d$ exactly, at about 12 times the cost of f, independent of n. **Read:** §8.2, pp. 215–216. (Not in the book.) `jvp(grad f)` in JAX, `torch.func` in PyTorch: Newton–CG without ever forming the Hessian. The CG iteration can be preconditioned as in Chapter 5.

Trust-region Newton–CG

Trust-region subproblem, with $B_k = \nabla^2 f_k$:

$$\min_{p \in \mathbb{R}^n} \; m_k(p) \stackrel{\text{def}}{=} f_k + (\nabla f_k)^T p + \tfrac{1}{2} p^T B_k p \qquad \text{subject to } \|p\| \le \Delta_k. \tag{7.11}$$

Algorithm 7.2 (Steihaug) is CG started from $z_0 = 0$, with three ways to stop:

1. $d_j^T B_k d_j \le 0$ (nonpositive curvature): move along d_j to the boundary;
2. $\|z_{j+1}\| \ge \Delta_k$: stop at the intersection with the boundary;
3. $\|r_{j+1}\| < \epsilon_k$: return z_{j+1}.

The step p_k is used in Algorithm 4.1. Near a well-behaved solution the bound is inactive, and the method reduces to the inexact Newton method of Theorems 7.1, 7.2.

Algorithm 7.2 (CG–Steihaug)

Given tolerance $\epsilon_k > 0$; set $z_0 = 0$, $r_0 = \nabla f_k$, $d_0 = -r_0 = -\nabla f_k$
if $\|r_0\| < \epsilon_k$ **return** $p_k = z_0 = 0$
for $j = 0, 1, 2, \ldots$
  if $d_j^T B_k d_j \leq 0$
    find τ such that $p_k = z_j + \tau d_j$ minimizes $m_k(p_k)$ in (4.5)
    and satisfies $\|p_k\| = \Delta_k$; **return** p_k
  set $\alpha_j = r_j^T r_j / d_j^T B_k d_j$; $z_{j+1} = z_j + \alpha_j d_j$
  if $\|z_{j+1}\| \geq \Delta_k$
    find $\tau \geq 0$ such that $p_k = z_j + \tau d_j$ satisfies $\|p_k\| = \Delta_k$; **return** p_k
  set $r_{j+1} = r_j + \alpha_j B_k d_j$
  if $\|r_{j+1}\| < \epsilon_k$
    return $p_k = z_{j+1}$
  set $\beta_{j+1} = r_{j+1}^T r_{j+1} / r_j^T r_j$; $d_{j+1} = -r_{j+1} + \beta_{j+1} d_j$

Theorem 7.3: two properties of Algorithm 7.2

- The initialization $z_0 = 0$ is crucial. If $\|\nabla f_k\|_2 \geq \epsilon_k$, then $m_k(p_k) \leq m_k(p_k^C)$: the step is at least as good as the Cauchy point, so the method is globally convergent.

Theorem 7.3

The sequence of vectors $\{z_j\}$ generated by Algorithm 7.2 satisfies

$$0 = \|z_0\|_2 < \cdots < \|z_j\|_2 < \|z_{j+1}\|_2 < \cdots < \|p_k\|_2 \leq \Delta_k.$$

- It is therefore acceptable to stop as soon as the boundary is reached: no later iterate with a lower value of m_k lies inside the trust region.
- **Read:** proof of Theorem 7.3 (uses Theorem 5.2), pp. 172–173.

Preconditioning the trust-region Newton–CG method

Nonsingular D; the trust region is defined in the scaled norm:

$$\min_{p\in\mathbb{R}^n} \; m_k(p) \stackrel{\text{def}}{=} f_k + \nabla f_k^T p + \tfrac{1}{2} p^T B_k p \qquad \text{subject to } \|Dp\| \le \Delta_k. \tag{7.13}$$

Change of variables $\hat{p} = Dp$, as in (5.37)–(5.38) with D in the role of C:

$$\nabla f_k^T p = (D^{-T}\nabla f_k)^T \hat{p}, \qquad p^T B_k p = \hat{p}^T (D^{-T} B_k D^{-1}) \hat{p}, \qquad \|Dp\| = \|\hat{p}\|.$$

The preconditioned subproblem, with $\hat{g}_k = D^{-T}\nabla f_k$ and $\hat{B}_k = D^{-T} B_k D^{-1}$:

$$\min_{\hat{p}\in\mathbb{R}^n} \; \hat{m}_k(\hat{p}) = f_k + \hat{g}_k^T \hat{p} + \tfrac{1}{2}\hat{p}^T \hat{B}_k \hat{p} \qquad \text{subject to } \|\hat{p}\| \le \Delta_k.$$

This has exactly the form (7.11): Algorithm 7.2 applies with $(\hat{g}_k, \hat{B}_k)$; $p_k = D^{-1}\hat{p}_k$. CG speed is now set by the eigenvalues of $\hat{B}_k$. $D = L^T$ from incomplete Cholesky, shifted by Algorithm 7.3 ($LL^T = \bar{B} + \alpha_k I$) if $\nabla^2 f_k$ is indefinite. **Read:** pp. 174–175.

Trust-region Newton–Lanczos

Algorithm 7.2 accepts *any* direction of negative curvature, even when it gives an insignificant reduction in the model. Example:

$$\min_p \; m(p) = 10^{-3}p_1 - 10^{-4}p_1^2 - p_2^2 \qquad \text{subject to } \|p\| \le 1.$$

Steepest descent at $p = 0$ is $(-10^{-3}, 0)^T$: reduction about 10^{-3}. Along e_2: about 1.

Lanczos instead of CG: after j steps, Q_j ($n \times j$, orthogonal columns spanning the Krylov subspace) with $Q_j^T B Q_j = T_j$ tridiagonal. Solve

$$\min_{w \in \mathbb{R}^j} \; f_k + e_1^T Q_j (\nabla f_k)\, e_1^T w + \tfrac{1}{2} w^T T_j w \qquad \text{subject to } \|w\| \le \Delta_k, \tag{7.14}$$

and set $p_k = Q_j w$. More robust; the subproblem is more expensive.

(Lanczos is not covered in this course; the example above is the point.)

Limited-memory quasi-Newton methods

For large problems whose Hessian cannot be computed at a reasonable cost or is not sparse. Recall the BFGS method (Algorithm 6.1):

$$x_{k+1} = x_k - \alpha_k H_k \nabla f_k, \tag{7.15}$$

$$H_{k+1} = V_k^T H_k V_k + \rho_k s_k s_k^T, \tag{7.16}$$

$$\rho_k = \frac{1}{y_k^T s_k}, \qquad V_k = I - \rho_k y_k s_k^T. \tag{7.17}$$

H_k is generally dense: storing it is prohibitive when n is large.

Idea of L-BFGS

Store only the m most recent pairs $\{s_i, y_i\}$, and discard the oldest pair at each iteration. Modest values of m (between 3 and 20, say) often work well.

The L-BFGS approximation

At iteration k the stored pairs are $\{s_i, y_i\}$, $i = k-m, \ldots, k-1$. Choose an initial approximation H_k^0 (it may vary from iteration to iteration) and apply (7.16) m times:

$$\begin{aligned} H_k = &\left(V_{k-1}^T \cdots V_{k-m}^T\right) H_k^0 \left(V_{k-m} \cdots V_{k-1}\right) \\ &+ \rho_{k-m}\left(V_{k-1}^T \cdots V_{k-m+1}^T\right) s_{k-m} s_{k-m}^T \left(V_{k-m+1} \cdots V_{k-1}\right) \\ &+ \rho_{k-m+1}\left(V_{k-1}^T \cdots V_{k-m+2}^T\right) s_{k-m+1} s_{k-m+1}^T \left(V_{k-m+2} \cdots V_{k-1}\right) \\ &+ \cdots \\ &+ \rho_{k-1} s_{k-1} s_{k-1}^T. \end{aligned} \tag{7.19}$$

H_k is never formed. Only the product $H_k \nabla f_k$ is computed, by a recursion.

Algorithm 7.4 (L-BFGS two-loop recursion)

$q \leftarrow \nabla f_k$
for $i = k-1, k-2, \ldots, k-m$
$\quad \alpha_i \leftarrow \rho_i s_i^T q; \qquad q \leftarrow q - \alpha_i y_i$
$r \leftarrow H_k^0 q$
for $i = k-m, k-m+1, \ldots, k-1$
$\quad \beta \leftarrow \rho_i y_i^T r; \qquad r \leftarrow r + s_i(\alpha_i - \beta)$
stop with result $H_k \nabla f_k = r$

Cost: $4mn$ multiplications, plus the product $H_k^0 q$ (n multiplications if H_k^0 is diagonal). Storage: the m pairs $\{s_i, y_i\}$.

The initial matrix H_k^0 and the line search

- In Algorithm 7.4 the multiplication by H_k^0 is isolated from the rest of the computation. H_k^0 can be chosen freely and can vary between iterations.
- A choice that has proved effective in practice: $H_k^0 = \gamma_k I$, where

$$\gamma_k = \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}}. \tag{7.20}$$

- γ_k is the scaling factor of (6.21): it estimates the size of the true Hessian along the most recent search direction. The search direction is then well scaled, and $\alpha_k = 1$ is accepted in most iterations.
- The line search should be based on the Wolfe conditions (3.6) or the strong Wolfe conditions (3.7), so that BFGS updating is stable.

Algorithm 7.5 (L-BFGS)

Choose starting point x_0, integer $m > 0$; $k \leftarrow 0$

repeat

choose H_k^0 (for example, by using (7.20))

compute $p_k \leftarrow -H_k \nabla f_k$ from Algorithm 7.4

compute $x_{k+1} \leftarrow x_k + \alpha_k p_k$, where α_k satisfies the Wolfe conditions

if $k > m$: discard the vector pair $\{s_{k-m}, y_{k-m}\}$ from storage

compute and save $s_k \leftarrow x_{k+1} - x_k$, $y_k = \nabla f_{k+1} - \nabla f_k$; $k \leftarrow k + 1$

until convergence

During its first $m - 1$ iterations, Algorithm 7.5 is equivalent to the BFGS algorithm of Chapter 6, if the initial matrix H_0 is the same and L-BFGS chooses $H_k^0 = H_0$.

Performance of L-BFGS (Table 7.1)

		$m = 3$		$m = 5$		$m = 17$		$m = 29$	
Problem	n	nfg	time	nfg	time	nfg	time	nfg	time
DIXMAANL	1500	146	16.5	134	17.4	120	28.2	125	44.4
EIGENALS	110	821	21.5	569	15.7	363	16.2	168	12.5
FREUROTH	1000	>999	—	>999	—	69	8.1	38	6.3
TRIDIA	1000	876	46.6	611	41.4	531	84.6	462	127.1

- nfg: function and gradient evaluations. Larger m: fewer evaluations, but each iteration costs more. The best m is problem dependent.
- The rate of convergence is linear. Main weakness: slow convergence on ill-conditioned problems (Hessian with a wide distribution of eigenvalues).

Relationship with conjugate gradient methods

Hestenes–Stiefel direction (5.46), with $s_k = \alpha_k p_k$:

$$p_{k+1} = -\nabla f_{k+1} + \frac{\nabla f_{k+1}^T y_k}{y_k^T p_k} p_k = -\Big(I - \frac{s_k y_k^T}{y_k^T s_k}\Big)\nabla f_{k+1} \equiv -\hat{H}_{k+1}\nabla f_{k+1}. \qquad (7.21)$$

$\hat{H}_{k+1}$ is neither symmetric nor positive definite. Symmetric, positive definite, and satisfying the secant equation:

$$H_{k+1} = \Big(I - \frac{s_k y_k^T}{y_k^T s_k}\Big)\Big(I - \frac{y_k s_k^T}{y_k^T s_k}\Big) + \frac{s_k s_k^T}{y_k^T s_k}. \qquad (7.22)$$

- (7.22) is one BFGS update of the identity: memoryless BFGS ($m = 1$, $H_k^0 = I$).
- Exact line search: $p_{k+1} = -H_{k+1}\nabla f_{k+1}$ is the Hestenes–Stiefel direction (7.23).

Theorem 7.4: compact representation of BFGS updating

Theorem 7.4

Let B_0 be symmetric and positive definite, and assume that the k vector pairs $\{s_i, y_i\}_{i=0}^{k-1}$ satisfy $s_i^T y_i > 0$. Let B_k be obtained by applying k BFGS updates with these vector pairs to B_0, using the formula (6.19). We then have that

$$B_k = B_0 - \begin{bmatrix} B_0 S_k & Y_k \end{bmatrix} \begin{bmatrix} S_k^T B_0 S_k & L_k \\ L_k^T & -D_k \end{bmatrix}^{-1} \begin{bmatrix} S_k^T B_0 \\ Y_k^T \end{bmatrix}, \tag{7.24}$$

where $S_k = [s_0, \ldots, s_{k-1}]$ and $Y_k = [y_0, \ldots, y_{k-1}]$ are $n \times k$ matrices (7.25), while L_k and D_k are the $k \times k$ matrices

$$(L_k)_{i,j} = \begin{cases} s_{i-1}^T y_{j-1} & \text{if } i > j, \\ 0 & \text{otherwise,} \end{cases} \qquad D_k = \operatorname{diag}\left[s_0^T y_0, \ldots, s_{k-1}^T y_{k-1}\right]. \tag{7.26–7.27}$$

The limited-memory compact form

Keep the m most recent pairs, $S_k = [s_{k-m}, \ldots, s_{k-1}]$, $Y_k = [y_{k-m}, \ldots, y_{k-1}]$ (7.28), and take the basic matrix $B_k^0 = \delta_k I$ with $\delta_k = 1/\gamma_k$:

$$B_k = \delta_k I - \begin{bmatrix} \delta_k S_k & Y_k \end{bmatrix} \begin{bmatrix} \delta_k S_k^T S_k & L_k \\ L_k^T & -D_k \end{bmatrix}^{-1} \begin{bmatrix} \delta_k S_k^T \\ Y_k^T \end{bmatrix}. \tag{7.29}$$

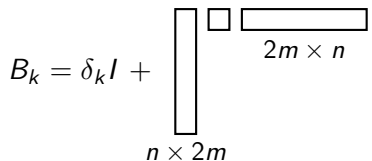


Figure 7.1

- The middle matrix is only $2m \times 2m$.
- Update: about $2mn + O(m^3)$ operations. Product $B_k v$: $(4m+1)n + O(m^2)$.
- Used in trust-region methods, L-BFGS-B, KNITRO, IPOPT.

(Not in the book.) IPOPT: `hessian_approximation=limited-memory` is (7.29).

SR1 in compact form; unrolling the update

SR1. With S_k, Y_k, D_k, L_k as in (7.25)–(7.27), k SR1 updates (6.24) of B_0 give

$$B_k = B_0 + (Y_k - B_0 S_k)\big(D_k + L_k + L_k^T - S_k^T B_0 S_k\big)^{-1}(Y_k - B_0 S_k)^T. \qquad (7.30)$$

Limited-memory SR1 is sometimes not as effective as L-BFGS: it may not produce positive definite approximations near a solution.

Unrolling. The obvious implementation of limited-memory BFGS for B_k:

$$B_k = B_k^0 + \sum_{i=k-m}^{k-1} \big[b_i b_i^T - a_i a_i^T\big], \qquad (7.33)$$

with a_i, b_i from Procedure 7.6: about $\frac{3}{2}m^2 n$ operations, against $2mn$ for the compact form.

Sparse quasi-Newton updates

Demand that B_k has the same sparsity pattern as the true Hessian:

$$\Omega \stackrel{\text{def}}{=} \{(i,j) \mid [\nabla^2 f(x)]_{ij} \neq 0 \text{ for some } x \text{ in the domain of } f\}.$$

$$\min_B \quad \|B - B_k\|_F^2 = \sum_{(i,j)\in\Omega} \left[B_{ij} - (B_k)_{ij}\right]^2, \tag{7.34a}$$

$$\text{subject to} \quad Bs_k = y_k, \quad B = B^T, \quad \text{and} \quad B_{ij} = 0 \text{ for } (i,j) \notin \Omega. \tag{7.34b}$$

- B_{k+1} is not guaranteed to be positive definite.
- Not scale invariant; disappointing in practice. **Read:** pp. 185–186.

Partially separable functions

f is a sum of **element functions**, each depending on only a few components of x:

$$f(x) = \sum_{i=1}^{ne} f_i(x). \tag{7.35}$$

Example, with four element functions $f_1, \ldots, f_4$:

$$f(x) = (x_1 - x_3^2)^2 + (x_2 - x_4^2)^2 + (x_3 - x_2^2)^2 + (x_4 - x_1^2)^2. \tag{7.36}$$

f_1 depends on the *element variables* $x_{[1]} = (x_1, x_3)^T = U_1 x$, where U_1 is a 2×4 *compactifying matrix*. With $\phi_1(z_1, z_2) = (z_1 - z_2^2)^2$, $f_1(x) = \phi_1(U_1 x)$ and

$$\nabla f_1(x) = U_1^T \nabla \phi_1(U_1 x), \qquad \nabla^2 f_1(x) = U_1^T \nabla^2 \phi_1(U_1 x)\, U_1. \tag{7.37}$$

Quasi-Newton updating, element by element

Key idea. Do not approximate $\nabla^2 f_1$ ($n \times n$). Maintain a small dense approximation $B_{[1]}$ of $\nabla^2 \phi_1$ (2×2 in the example). After a step from x to x^+:

$$s_{[1]} = x^+_{[1]} - x_{[1]}, \qquad y_{[1]} = \nabla \phi_1(x^+_{[1]}) - \nabla \phi_1(x_{[1]}), \tag{7.38}$$

and $B_{[1]}$ is updated by BFGS or SR1. The same is done for every ϕ_i. Then

$$f(x) = \sum_{i=1}^{ne} \phi_i(U_i x) \quad (7.40), \qquad\qquad B = \sum_{i=1}^{ne} U_i^T B_{[i]} U_i \quad (7.41).$$

A trust-region step solves $B_k p_k = -\nabla f_k$ (7.42) approximately by CG. B_k is not assembled: the products $B_k v$ are computed from the U_i and $B_{[i]}$.

Partially separable updating: why it works, and its limits

- A problem of the form (7.36) with 1000 variables: each $B_{[i]}$ is 2×2 and becomes accurate after just a few iterations. Standard BFGS or L-BFGS, which ignore the structure, need many more iterations.
- The curvature condition $s_{[i]}^T y_{[i]} > 0$ is not guaranteed: some $\nabla^2 \phi_i$ may be indefinite. Remedy: SR1 updates of the element Hessians (LANCELOT).
- Limitations: the cost of the step computation (7.42) is comparable to the cost of a Newton step, and the partially separable structure can be difficult to identify.
- When the structure is known, it may be more efficient to compute a Newton step. **Read:** pp. 188–189.

Chapter 7 in one table

Method		Uses	Main property	Software (§7.5)
Newton–CG, line search	Alg. 7.1	$\nabla^2 f_k d$	rate set by η_k	TN/TNBC, TNPACK
Newton–CG, trust region	Alg. 7.2	$\nabla^2 f_k d$	$\geq$ Cauchy decrease	LANCELOT, TRON
Newton–Lanczos	(7.14)	$\nabla^2 f_k d$	more robust	GLTR
L-BFGS	Alg. 7.5	∇f	linear rate	LBFGS, M1QN3
Compact B_k	(7.29)	∇f	B_k instead of H_k	L-BFGS-B, IPOPT
Partially separable	(7.41)	structure	rapid, robust	LANCELOT

(Not in the book.) In SciPy, `scipy.optimize.minimize` offers the methods `Newton-CG`, `trust-ncg`, `trust-krylov` and `L-BFGS-B`.

Three results to carry forward

1. **The forcing sequence controls the rate.** With $\|r_k\| \le \eta_k \|\nabla f_k\|$: $\eta_k \to 0$ gives superlinear and $\eta_k = O(\|\nabla f_k\|)$ quadratic convergence. Only Hessian–vector products are needed.
2. **CG–Steihaug.** Started from $z_0 = 0$, the iterates grow in norm and the first step is the Cauchy step. The iteration stops at the trust-region boundary or at a direction of nonpositive curvature.
3. **L-BFGS.** $H_k \nabla f_k$ is computed from the m most recent pairs $\{s_i, y_i\}$ by the two-loop recursion, at $4mn$ multiplications. H_k is never formed.

Proved in the book: Theorem 7.3. Informal derivation: Theorems 7.1, 7.2.
Stated without proof: Theorem 7.4.

Homework (problem 3 is not in the book)

1. Exercise 7.2: show that the matrix $\hat{H}_{k+1}$ in (7.21) is singular.
2. Exercise 7.3: derive the formula (7.23) under the assumption that line searches are exact.
3. By hand. $m = 1$, $s_{k-1} = (1, 0)^T$, $y_{k-1} = (2, 1)^T$, $\nabla f_k = (1, 1)^T$, and $H_k^0 = \gamma_k I$ from (7.20). Run Algorithm 7.4. Check the result against $H_k \nabla f_k$ with H_k from (7.16).
4. Exercise 7.5: write $f(x) = x_2 x_3 e^{x_1 + x_3 - x_4} + (x_2 x_3)^2 + (x_3 - x_4)$ in the form (7.40). Give each compactifying matrix U_i.
5. Exercise 7.1: code Algorithm 7.5 and test it on the extended Rosenbrock function (p. 191) from $(-1, -1, \ldots, -1)^T$. Observe the behavior for various values of m.